SDU

General information
printf()

Course outline

- **The plan is indicative, subject to change!**

Date	Lecture	Subject(s)	Lab	Assignment due
Feb 2	1	Introduction; ARM Cortex-M4	Lab1: Setting up the development env.	
Feb 9	2	EMP coding standard; Bit manipulation	Lab2: Bit manipulation	
Feb 16	3	State machines; The compiler chain; EMP-Board	Lab3: State machines	PF1 – Assignment 1
Feb 23	4	The task model; The pre-processor	Lab4: A clock radio task model	PF1 – Assignment 2
Mar 2	5	Queues and semaphores; Debugging	Lab5: Debug with a serial connection	PF1 – Assignment 3
Mar 9	6	Run to complete scheduler	Lab6: RTCS, Run to compile scheduler	PF1 – Assignment 4
Mar 16	7	More debugging; C: printf()	Lab7: RTCS, Debugger	PF1 – Assignment 5
March 23	8	FreeRTOS	Lab8: FreeRTOS	PF1 – Assignment 6
March 30 April 6		Easter holiday		
Apr 13	9	More queues; Assembler in C	Lab9: FreeRTOS (continued)	PF1 – Assignment 7
Apr 20	10	Re-entrance		PF1 – Assignment 8
Apr 27	11	Work on the final assignment, consultations		
May 4	12	Poster session		PF2- Final Assignment

Today’s agenda

- Assignment05 presentation
- Organizational issues
- Course feedback
- Recap
- Lecture
  - printf()
- Lab07 / Assignment06

SDU

Exam

- Condition:
  - completion of two assignments: one during the semester and the final assignment
- For electronics students: you will figure out if you are taking my exam or Operating Systems one day before the exam at 8:00
  - The assignment of students to the two exams will be random
- For robotek and exchange students: you are taking my exam
- Exchange students can now take the exam between June 10-12 if they want to

SDU

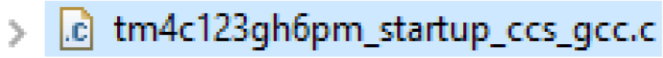
Exam

- Format:
  - Individual in-person oral examination
  - You draw one question out of 4: state machines (and material covered before), RTCS, RTOS, final project
  - Common topics: queues, semaphores, mutexes
  - I can also ask any other questions (e.g. final assignment task diagram)
  - Short presentation (2-3min) of drawn question (possible but not necessary) counts towards the grade
  - Q&A with me and external censor
  - About 15 minutes all together
  - Danish grading system
  - Grade dependent of exam performance (which can be influenced by final assignment solution quality)

SDU

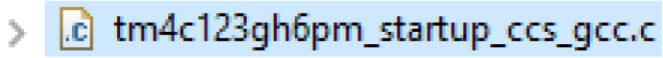
Recap – how to start a project

- Itslearning –> Resources -> Notes -> CreateAProjectInCCS.pdf
- Copy any needed/provided files to the project
- Set up the systick handler!
  - Open the following file which was created automatically when you created the project



Recap – how to start a project

- Itslearning –> Resources -> Notes -> CreateAProjectInCCS.pdf
- **Copy any provided files to the project folder**
- Set up the systick handler!
  - Open the following file which was created automatically when you created the project



Set up handler

- Add the following lines to enable the interrupt handler
- Compile and run
- A blue light should be blinking on your board

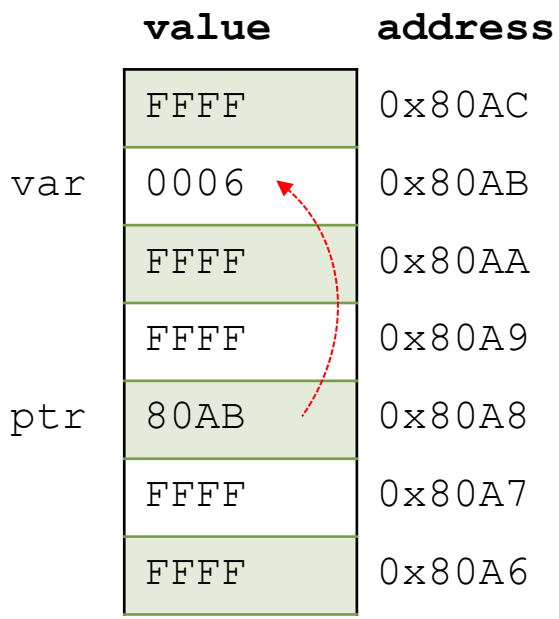
```
void ResetISR(void);
static void NmiSR(void);
static void FaultISR(void);
static void IntDefaultHandler(void);

void systick_handler(void);

#ifndef HWREG
#define HWREG(x) (*((volatile uint32_t *)(x)))
#endif

//*****************************************************************************
//
// The entry point for the application.
//
//*****************************************************************************
extern int main(void);

//*****************************************************************************
//
// Reserve space for the system stack.
//
//*****************************************************************************
static uint32_t pui32Stack[128];

//*****************************************************************************
//
// External declarations for the interrupt handlers used by the application.
//
//*****************************************************************************
// To be added by user

//*****************************************************************************
//
// The vector table.  Note that the proper constructs must be placed on this to
// ensure that it ends up at physical address 0x0000.0000 or at the start of
// the program if located at a start address other than 0.
//
//*****************************************************************************
__attribute__ ((section(".intvecs")))
void (* const g_pfnVectors[])(void) =
{
    (void (*)(void))((uint32_t)pui32Stack + sizeof(pui32Stack)),
                                            // The initial stack pointer
    ResetISR,                               // The reset handler
    NmiSR,                                  // The NMI handler
    FaultISR,                               // The hard fault handler
    IntDefaultHandler,                      // The MPU fault handler
    IntDefaultHandler,                      // The bus fault handler
    IntDefaultHandler,                      // The usage fault handler
    0,                                      // Reserved
    0,                                      // Reserved
    0,                                      // Reserved
    0,                                      // Reserved
    IntDefaultHandler,                      // SVCall handler
    IntDefaultHandler,                      // Debug monitor handler
    0,                                      // Reserved
    IntDefaultHandler,                      // The PendSV handler
    systick_handler,                        // The SysTick handler
    IntDefaultHandler,                      // GPIO Port A
```

Systick.c

```c
#include <stdint.h>
#include "tm4c123gh6pm.h"



#define SYSTICK_RELOAD_VALUE 80000              // 5 mS

// Missing definitions in tm4c123gh6pm.h file
#define NVIC_INT_CTRL_PEND_SYST   0x04000000    // Pend a systick int
#define NVIC_INT_CTRL_UNPEND_SYST 0x02000000    // Unpend a systick int

#define SYSTICK_PRIORITY    0x7E

volatile int ticks = 0;

void systick_handler(void)
/****************************************************************************
*   Function : See module specification (.h-file).
****************************************************************************/
{
  // Hardware clears systick int reguest
  ticks++;
}

```

Recap - Variables and Pointers

```
INT16S var;
INT16S *ptr // pointer to INT16S
var = 6;
ptr = &var; // ptr = adr of var
*ptr = 8; // what ptr is pointing to
```



Recap - Pointers to functions

```
int (*p_funk)(int,int);
if( wanted_task == TASK_1 )
      p_funk = task_1;
else
      p_funk = task_2;
result = (*p_task)(1,100);
```


SDU

Recap - Pointers to functions

```
typedef struct
// task control block
{
  INT8U  condition;
  INT8U  name;
  INT8U  state;
  INT8U  event;
  INT8U  sem;
  INT16U timer;
  void (*tf)(INT8U, INT8U, INT8U, INT8U);
// Pointer to task function
} tcb;
```

Assignment 4

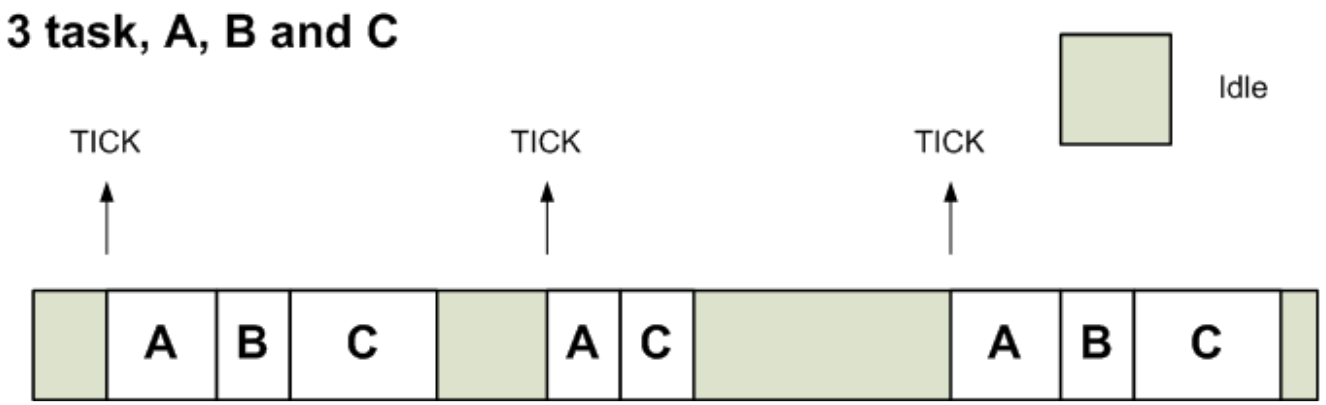
```
int main(void)
{
        init_gpio();
        init_rtcs();

        open_queue( Q_UART_TX );
        open_queue( Q_UART_RX );
        open_queue( Q_LCD );

        start_task( TASK_UART_TX, uart_tx_task );
        start_task( TASK_UART_RX, uart_rx_task );
        start_task( TASK_UI, ui_task );
        start_task( TASK_LCD, lcd_task );
        start_task( TASK_RTC, rtc_task );
        start_task( TASK_DISPLAY_RTC, display_rtc_task );

        schedule();

        return(0);
}
```

Assignment 4

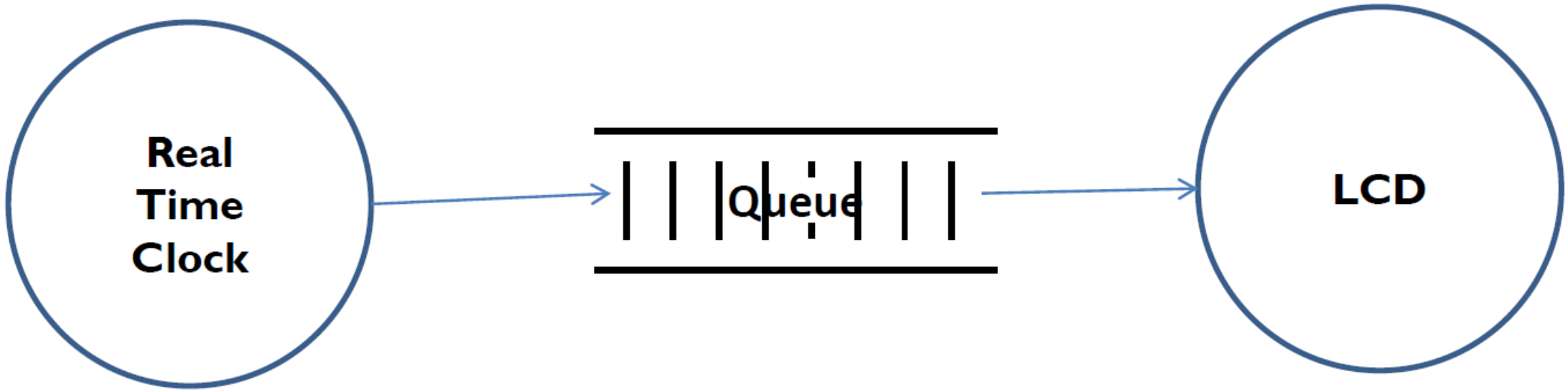
```
if( pot[current_task].condition == TASK_READY )                    // if the task is ready
    pot[current_task].tf(current_task, pot[current_task].state, pot[current_task].event, 0);    // run the task function
current_task++;                                                    // increment the task counter (go to next task)
} while ( pot[current_task].condition != TASK_IDLE );
```

Recap of RTCS

- Run to Complete Scheduling
  - Non-preemptive scheduling
  - Each task runs until it is finished
  - Concept not complex – implementation might be
  - Superloop now within the schedule() function



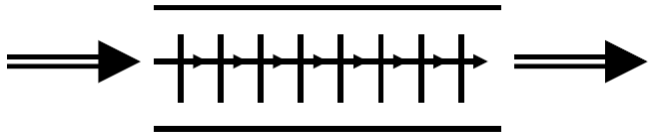
Recap Producer – consumer model



”12:31:55”
100nS/char

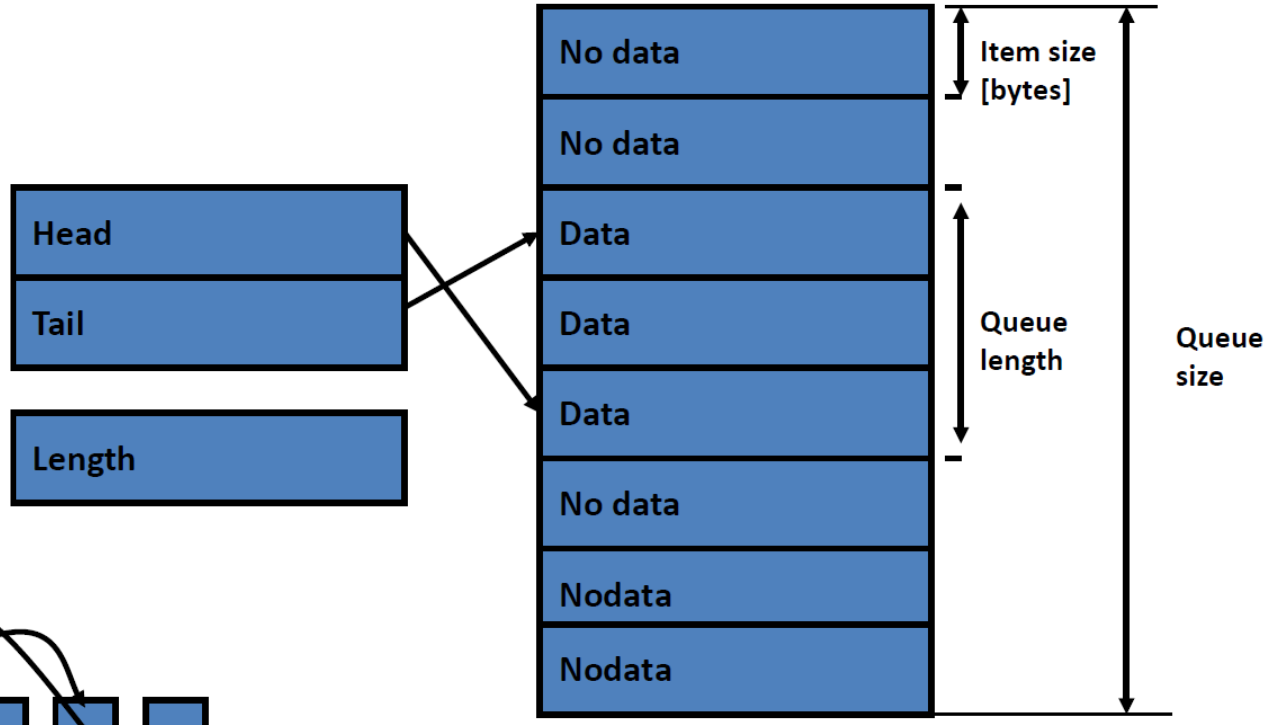
”12:31:55”
1mS/char

Recap - Queue implementation

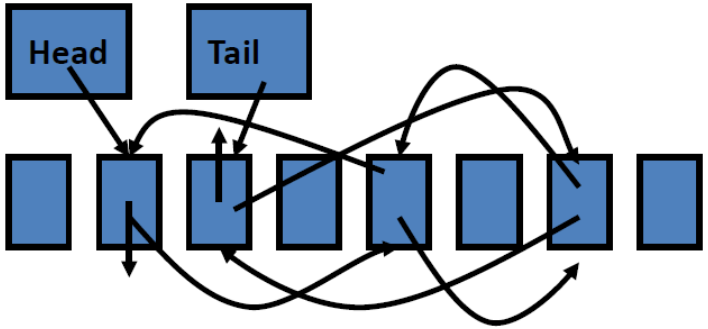


1) Move data through the queue (like supermarket)

2) Use pointers (pull a number
-like at the post office)

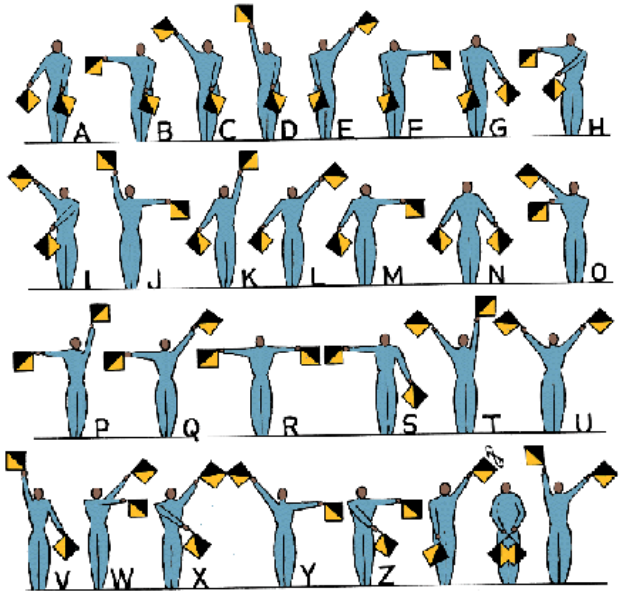


3) Linked List (like exam
– call the next)



SDU

Recap Semaphores

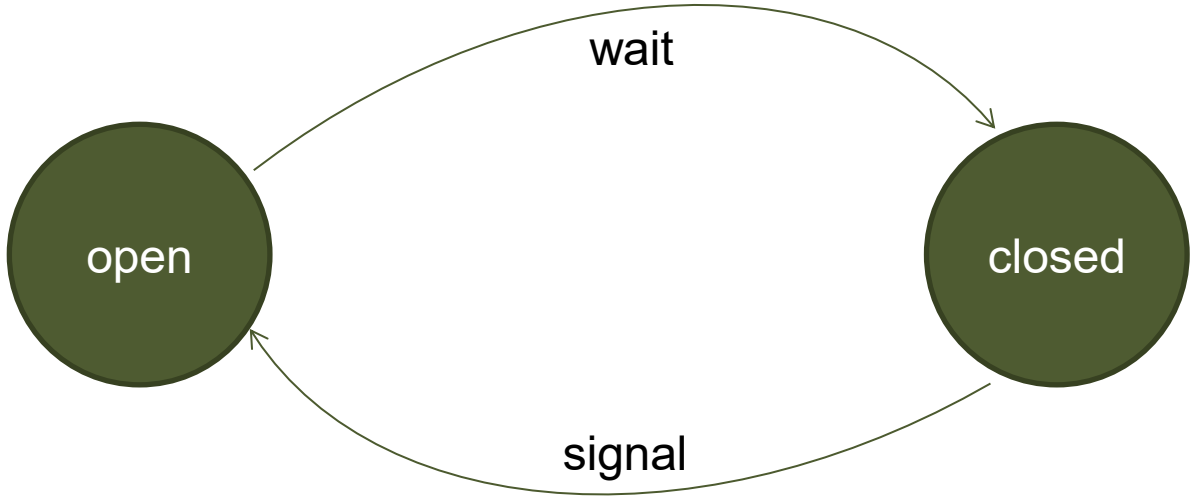


- In real life: visual signaling with arms & flags
- In software: data structure for synchronization
- It's an integer, but
  - The only allowed operations are increment and decrement by one
  - When a task decrements the semaphore, if the result is negative, the task blocks itself and cannot continue until another task increments the semaphore
  - When a task increments the semaphore, if there are other tasks waiting, one of the waiting tasks gets unblocked.
- Counter or binary
- Example parking access control

Recap Semaphore functions

Increment() ⇔ signal() ⇔ V()
Decrement() ⇔ wait() ⇔ P()

increment_and_wake_a_waiting_process_if_any ()
decrement_and_block_if_the_result_is_negative ()



SDU

Recap Semaphore purposes

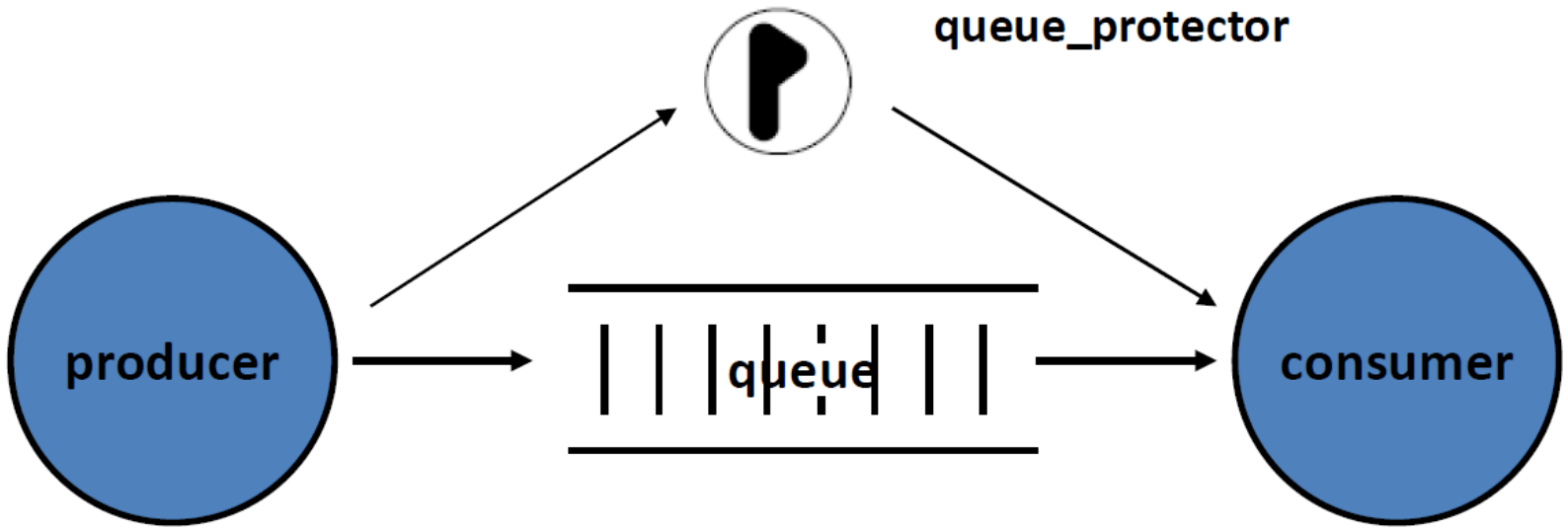
- Signaling - serialization
- Mutex – mutual exclusion

Recap Semaphores – code

```
BOOLEAN wait( INT8U sem_no)
{
        INT8U Result;

        if( sem_no< MAX_SEM )
        {
                Result = semaphore[sem_no];
                semaphore[sem_no] = FALSE;
        }
        else
                Result = FALSE;
        return( Result );
}

void signal( INT8U sem_no)
{
        if( sem_no< MAX_SEM )
                semaphore[sem_no] = TRUE;
}
```

Recap Protection of queues

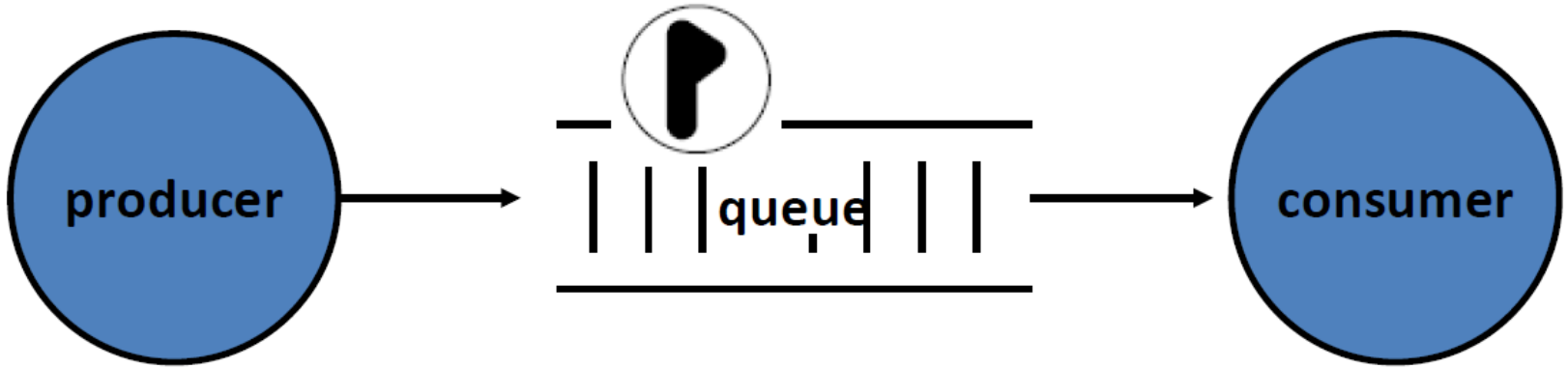
A queue is a shared resource, and as such, it must be protected by a mutex semaphore.



SDU

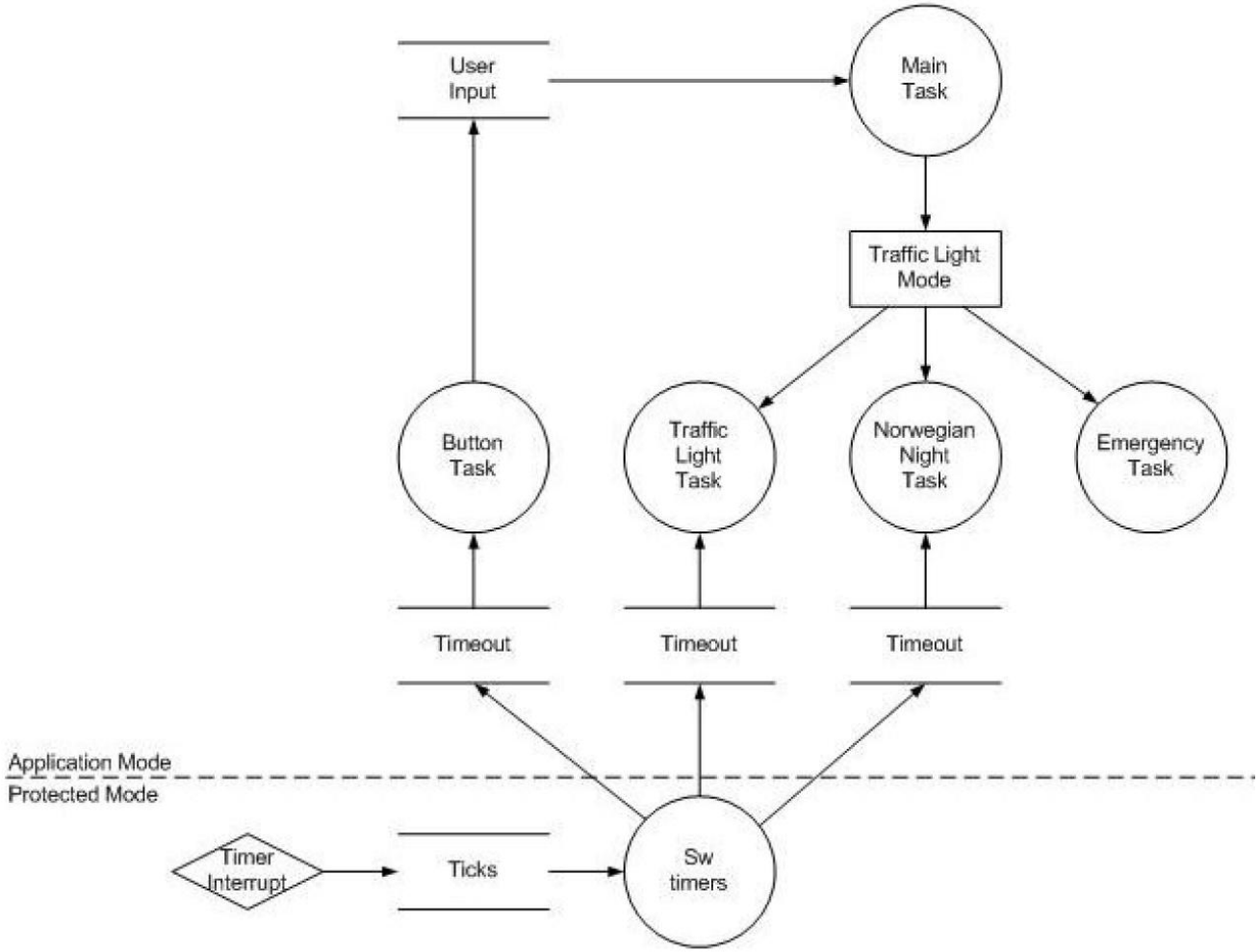
Recap Protection of queues

Because of that, the semaphore is often built into the queue or closely connected to the queue.



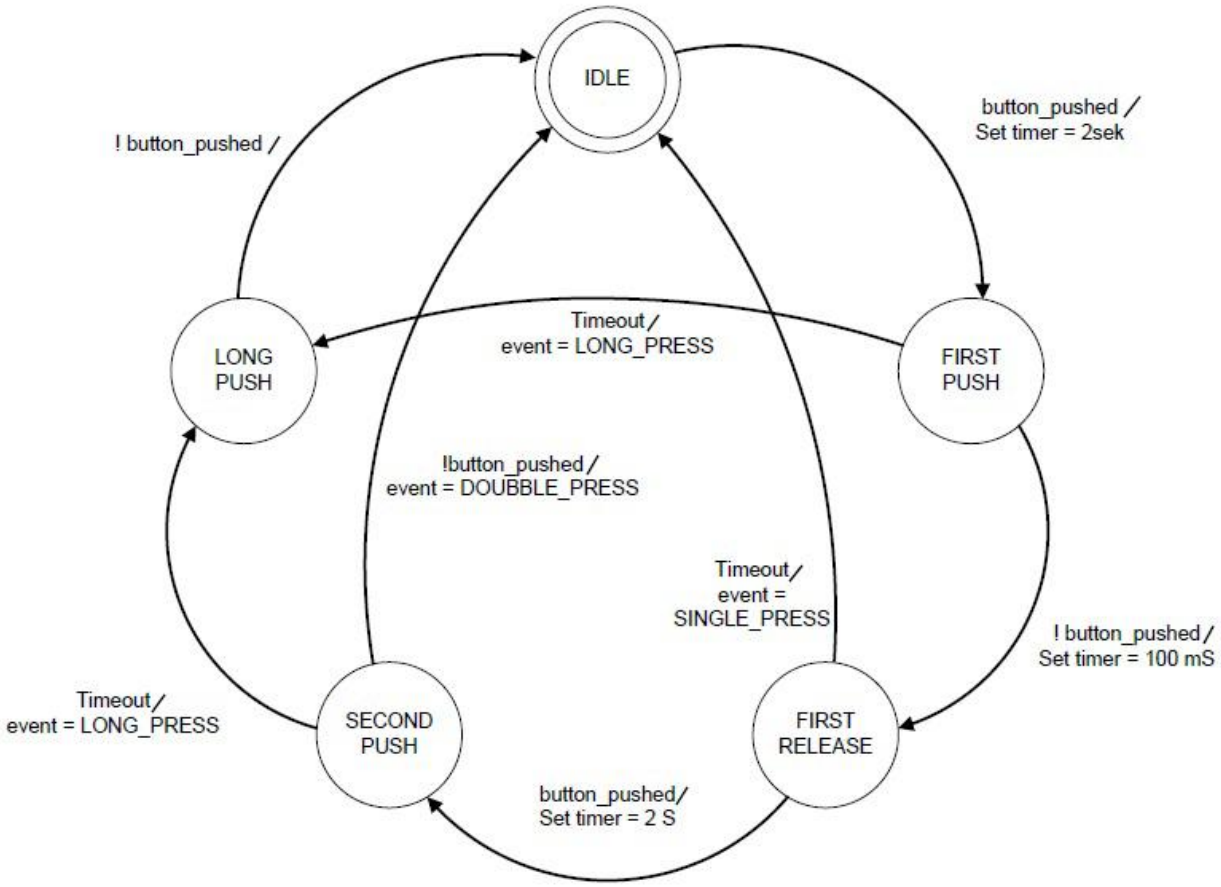
SDU

Recap Task Diagrams




```
  init_systick();
  init_gpio();

  // Loop forever.
  while(1)
  {
  // System part of the super loop.
  // ----------------------------
  while( !ticks );


  // The following will be executed every 5ms
  ticks--;

  if( ! --alive_timer )
  {
    alive_timer = MILLISEC( 500 );
    GPIO_PORTD_DATA_R ^= 0x40;
  }

    // Protected operating system mode
    swt_ctrl();                                // task controlling the timers

    // Application mode
    button_task( TASK_BUTTON );
    main_task( TASK_MAIN );
    tl_task( TASK_TRAFFIC_LIGHT );
    flash_task( TASK_NORWEGIAN_NIGHT );
    red_task( TASK_EMERGENCY );

  }
}
```

Recap Button Task




```
void button_task( INT8U task_no )

/*****************************************************************************
*   Input    :
*   Output   :
*   Function :
*****************************************************************************/
{
  static INT8U  button_state = BS_IDLE;
         INT8U  time_out;

  time_out = get_msg_event( SEB_TO_BUTTON );

  switch( button_state )
  {
    case BS_IDLE:
      if( button_pushed( ))                                       // if button pushed
      {
          button_state = BS_FIRST_PUSH;                              // we go from the idle state to first push
          start_swtimer( ST_BUTTON, SEB_TO_BUTTON, MILLISEC(2000) );  // we set a timer of 2s to see if this is a long push
      }
      break;
    case BS_FIRST_PUSH:
      if( time_out )                                              // if the timer runs out before the button was released it was a long push
      {
          button_state = BS_LONG_PUSH;
          put_msg_event( SEB_BUTTON_EVENT, BE_LONG_PUSH );
      }
      else
      {
        if( !button_pushed() )                                        // if button released before the timer runs out it was a normal push
    {
           button_state = BS_FIRST_RELEASE;
          start_swtimer( ST_BUTTON, SEB_TO_BUTTON, MILLISEC(100) );   // we set a new timer to see if the button is pressed again shortly after (double push)
        }
      }
      break;
    case BS_FIRST_RELEASE:
      if( time_out )                                              // if the timer runs out without the button being pushed again it was a single push
      {
          button_state = BS_IDLE;
          put_msg_event( SEB_BUTTON_EVENT, BE_SINGLE_PUSH );
      }
      else
      {
        if( button_pushed() )                                       // if button is pressed again before the timer runs out it was a double push
    {
          button_state = BS_SECOND_PUSH;
          start_swtimer( ST_BUTTON, SEB_TO_BUTTON, MILLISEC( 2000 )); // we set a new timer to see if it is a long push
        }
      }
      break;
    case BS_SECOND_PUSH:
      if( time_out )                                              // if the timer runs out before button is released it was a long push
      {
        button_state = BS_LONG_PUSH;
        put_msg_event( SEB_BUTTON_EVENT, BE_LONG_PUSH );
        }
      else
      {
        if( !button_pushed() )                                  // if button released before the timer runs out it was a double push
    {
          button_state = BS_IDLE;
          put_msg_event( SEB_BUTTON_EVENT, BE_DOUBBLE_PUSH );
        }
      }
      break;
    case BS_LONG_PUSH:
        if( !button_pushed() )                                  // when the button is released after a long push we go back to the idle state
            button_state = BS_IDLE;
      break;
    default:
```

printf()

- The most famous and strange C function
- Print formatted data
- Write data according to format string
- After format – as many additional arguments as specified in format

```
int printf ( const char * format, ... );
```


SDU

printf() parameters

```
printf("The timer and state values are: %d %d ", timer, state);
```

- Format
  - The string that contains the text to be written
  - It can optionally contain embedded format tags that are substituted by the values specified in subsequent argument(s) and formatted as requested.
  - The number of arguments following the *format* parameters should at least be as many as the number of format tags.
  - The format tags follow this prototype:

%[flags][width][.precision][length]specifier

  - Where specifier is the most significant one and defines the type and the interpretation of the value of the corresponding argument:

printf() specifiers

specifier	Datatype	Example
c	Character	a
d or i	Signed decimal integer	392
e	Scientific notation (mantissa/exponent) using e character	3.9265e+2
E	Scientific notation (mantissa/exponent) using E character	3.9265E+2
f	Decimal floating point	392.65
g	Use the shorter of %e or %f	392.65
G	Use the shorter of %E or %f	392.65
o	Unsigned octal	610
s	String of characters	sample
u	Unsigned decimal integer	7235
x	Unsigned hexadecimal integer	7fa
X	Unsigned hexadecimal integer (capital letters)	7FA
p	Pointer address	B800:0000
n	Nothing printed. The argument must be a pointer to a signed int, where the number of characters written so far is stored.	
%	A % followed by another % character will write % to stdout.	%

SDU

printf() flags

flags	description
-	Left-justify within the given field width; Right justification is the default (see width sub-specifier).
+	Forces to precede the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a - sign.
(space)	If no sign is going to be written, a blank space is inserted before the value.
#	Used with o, x or X specifiers the value is preceeded with 0, 0x or 0X respectively for values different than zero. Used with e, E and f, it forces the written output to contain a decimal point even if no digits would follow. By default, if no digits follow, no decimal point is written. Used with g or G the result is the same as with e or E but trailing zeros are not removed.
0	Left-pads the number with zeroes (0) instead of spaces, where padding is specified (see width sub-specifier).

SDU

printf() width and precision

width	description
(number)	Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.
*	The width is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.

.precision	description
.number	For integer specifiers (d, i, o, u, x, X): precision specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A precision of 0 means that no character is written for the value 0. For e, E and f specifiers: this is the number of digits to be printed after the decimal point. For g and G specifiers: This is the maximum number of significant digits to be printed. For s: this is the maximum number of characters to be printed. By default all characters are printed until the ending null character is encountered. For c type: it has no effect. When no precision is specified, the default is 1. If the period is specified without an explicit value for precision, 0 is assumed.
.*	The precision is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.

printf() length

length	description
h	The argument is interpreted as a short int or unsigned short int (only applies to integer specifiers: i, d, o, u, x and X).
l	The argument is interpreted as a long int or unsigned long int for integer specifiers (i, d, o, u, x and X), and as a wide character or wide character string for specifiers c and s.
L	The argument is interpreted as a long double (only applies to floating point specifiers: e, E, f, g and G).

SDU

printf() examples

```
/* printfexample */
#include <stdio.h>
int main()
{
   printf("Characters: %c %c \n", 'a', 65);
   printf("Decimals: %d %ld\n", 1977, 650000L);
   printf("Preceding with blanks: %10d \n", 1977);
   printf("Preceding with zeros: %010d \n", 1977);
   printf("Some different radixes: %d %x %o %#x %#o
\n", 100, 100, 100, 100, 100);
   printf("floats: %4.2f %+.0e %E \n", 3.1416, 3.1416,
3.1416);
   printf("%s \n", "A string");
   return 0;
}
```

And here is the output:

```
Characters: a A
Decimals: 1977 650000
Preceding with blanks:      1977
Preceding with zeros: 0000001977
Some different radixes: 100 64 144 0x64 0144

floats: 3.14 +3e+000 3.141600E+000

A string
```

printf() variants

```
int printf(const char *format, ...)     // print formatted data to stdout

int sprintf(char *str, const char *format, ...)   // write data to string

int fprintf(FILE *stream, const char *format, ...) // write to stream (FILE)
```

EMP – string.c

```
const void gfprintf( FILE fp, const char *format, ...)
```

Supports %c, %d, %u, %x, %X, %p, %s and %%.
Works through a callback

```
void putc1( FILE *fp, INT8U ch )
{
   if( *fp < 0x20000000 ) // It is a file
   {
      put_file( *fp, ch );
   }
   else // It is a pointer to a string
   {
      **fp = ch;
      (*fp)++;
   }
}
```

Files, Streams and Pipelines

- A task may communicate by reading and writing files. A file can be:
  - A data set that you can read and write repeatedly.
  - A stream of bytes generated by a program (such as a pipeline).
  - A stream of bytes received from or sent to a peripheral device.

A file API may include:
- open_file
- close_file
- read_file
- write_file
- seek_file

SDU

EMP – file.c

```
typedef struct
{
   BOOLEAN (*put)(INT8U); // Pointer to device put function
   BOOLEAN (*get)(INT8U*); // Pointer to device get function
} fcb; //file control block

fcb pof[MAX_FILES]; // Pool of files

BOOLEAN put_file( FILE file, INT8U ch )
{
   if( pof[file].put )
      pof[file].put( ch );
}

BOOLEAN get_file( FILE file, INT8U *pch )
{
   if( pof[file].get )
      pof[file].get( pch );
}
```

EMP – file.c

```
externINT8U init_files()
/**************************************************
Function: See modulespecification(.h-file).
**************************************************/
{
   INT8U i;
   for( i = 0; i < MAX_FILES; i++ )
   {
      pof[i].put = NULL;
      pof[i].get= NULL;
   }
   pof[COM1].put = uart_put_q;
   pof[COM1].get= uart_get_q;
   pof[COM2].put = wr_ch_LCD;
   pof[COM3].get= get_keyboard;
   return( 1 );
}
```

EMP – file.h

```
typedef int FILE;
#define COM1 0 // UART
#define COM2 1 // LCD
#define COM3 2 // Keyboard
```

EMP – file example

```
void ui_key_task(INT8U my_id, INT8U my_state, INT8U event, INT8U data)
{
   INT8U ch;
   if( get_file( COM3, &ch ))
   {
      gfprintf( COM1, “You pressed: %c\r\n", ch );
      gfprintf( COM2, "%c%cYou pressed: %c", ESC, 0xc0, ch );
   }
}
```

General I/O interaction with devices

- Using files
- All i/o devices (keyboard, LCD, UART) are communicated with using files
- This does not involve GPIO, general purpose input/output pins of the controller
- Details about printf implementation can be found in string.c of the solution to A05